

Planned raw in simple :

Quick flag before the breakdown: several citations in the Gemini research (TDFlow, SlopCodeBench, SWE-CI, CODENS, the specific arXiv IDs) I can't verify — treat those as "claimed research, unconfirmed" not established fact. Also, the Jio-account-cycling / key-rotation-proxy scheme in Part 2 is a **ToS gray area** — having legitimate access to multiple accounts doesn't make pooling their quotas through a rotator compliant with Google's terms. I'll list it since you asked, but flag it as legitimacy risk, not "safe loophole."

PART 1 — AI Setup (habitual, needs ongoing prompting/maintenance)

This is the docs/AGENTS.md/context-engineering side — stuff Gemini/Claude read every session. **You already have ~60% of this** (AI_INDEX.md, docs/decisions, docs/architecture, docs/reference, AI_POLICY.md).

Phase 1 — Source of truth cleanup (you're mostly done)

- Business rules → `docs/reference/BUSINESS\_RULES.md`  exists
- Architecture → `docs/architecture/SYSTEM\_BLUEPRINT.md`  exists
- ADRs → `docs/decisions/DECISION\_LOG.md`  exists
- Gap: no `docs/domain/glossary.md` (defines "party," "credit," "financial event" etc. so agents stop inventing meaning)

Phase 2 — Shrink/audit AGENTS.md-equivalent

- Your `AI\_POLICY.md` is close to this role already
- Action: check it doesn't contain lint/naming rules that should live in ESLint/TSCConfig instead (both papers call this "Lint Leakage" — very common failure mode)

Phase 3 — Task contract template

- Every Gemini prompt should carry: Objective / Scope / Source-of-truth files / Forbidden changes / Acceptance criteria / Verification command / Stop condition
- This is a direct upgrade of your existing "two sequential prompts" pattern (verify-then-fix) — just formalize it as a template you paste each time

Phase 4 — Independent review step

- After Gemini implements, run a *second* pass (Claude, or a fresh Gemini session with no memory of the implementation) whose only job is: does this match spec / did it touch unrelated files / does `git diff` match the claim
- Matches your existing "require real command output as evidence" principle — just adds a second reviewer instead of trusting the implementer's own report

Phase 5 — Regression + doc-sync habit

- After any schema/architecture change: does a source-of-truth doc need updating? (You already do this per AI_POLICY.md sync rules)

- Add: a short regression checklist (`npm run build`, `tsc`, existing tests) run *before and after* each task, not just after

****Phase 6 — Periodic context audit****

- Every few weeks, skim AI_POLICY.md / DECISION_LOG.md for stale references or contradictions — same instinct as your docs audit, just recurring

PART 2 — Manual / Tools / Tokens (you set up once, outside prompting)

****Phase 1 — Compute access (verify what you actually have)****

- Confirm current Jio AI Pro / Gemini access status post the June export-control suspension/restoration (this changed recently — worth re-checking your quota tier directly rather than assuming)
- Decide explicitly: are you using one account or would you consider the multi-account rotation approach? ****I'd flag this rather than build it**** — pooling quota across cycled accounts is the part most likely to violate ToS even if each account is individually legitimate

****Phase 2 — Static analysis (cheap, no ambiguity, do this first)****

- ESLint + Prettier + strict TSConfig configured to *enforce* naming/formatting — this replaces a big chunk of "AI instructions" with tooling that can't be ignored or drift
- Prisma schema validation as part of your build step (you likely already run this)

****Phase 3 — Testing/regression infra****

- A minimal test suite (you have some — `tests/` folder) run as a gate before accepting Gemini's work, not after-the-fact narrative
- This is the highest-leverage manual investment per both papers

****Phase 4 — Repository indexing (optional, higher effort)****

- Tools like Graphify (AST-based codebase graph) are the kind of thing worth trying only after Phases 1–3 are solid — treat their specific performance claims (e.g. "71.5x") as unverified marketing numbers, not confirmed benchmarks
- Lower priority for a single-tenant ERP of your size; probably skip unless you hit real context-retrieval pain

****Phase 5 — Secrets/env hygiene****

- Single `.env` source of truth (you already avoid duplication) — a secrets manager (Doppler/Infisical) is optional overkill for a solo single-tenant project unless you start multi-environment deploys

****Suggested order to actually study/implement:**** Part 1 Phase 1–3 → Part 2 Phase 2–3 → Part 1 Phase 4–6 → everything else is optional/lower-priority for your project's scale.

Raw data :

This chatgtp :

Worked for 54s

Yes. I researched this specifically around your MBM problem: AI agents doing many tasks successfully while gradually introducing architectural drift, duplicate systems, inconsistent naming, missing schema pieces, sync bugs, stale documentation, and “it works” code that becomes harder to modify.

The important conclusion from the 2026 research is:

Do not try to make pure vibe coding better by writing better giant prompts. Build an engineering system around the agent.

The strongest current pattern is:

Specification → persistent context → constrained implementation → automated verification → independent review → regression/maintenance checks → update the source of truth.

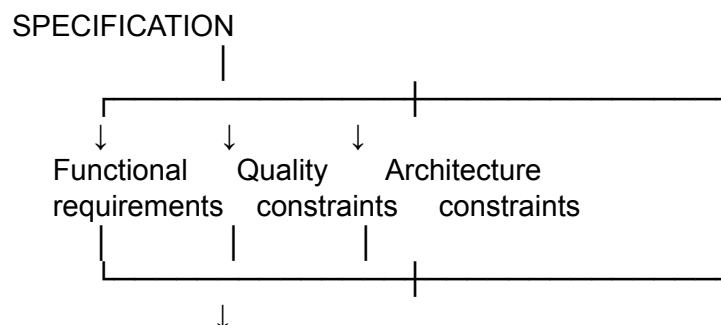
That is what I would build for MBM.

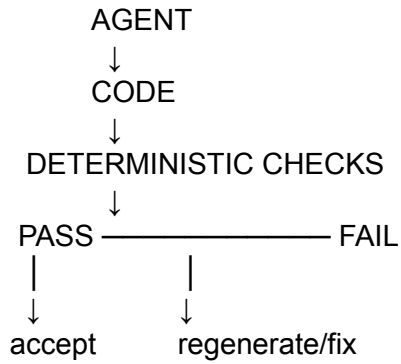
1. Specification as the source of truth — highest priority

The strongest recent paper for your question is Specification-Driven Development as the Foundation of AI-Native Enterprise Software Engineering, July 2026. It explicitly contrasts vibe coding with specification-driven development and proposes a model where structured specifications are the authoritative source, while generated code is a derived artifact that is continuously verified.

[Specification-Driven Development paper —
arXiv](https://arxiv.org/abs/2607.16680?utm_source=chatgpt.com)

The paper's model is essentially:



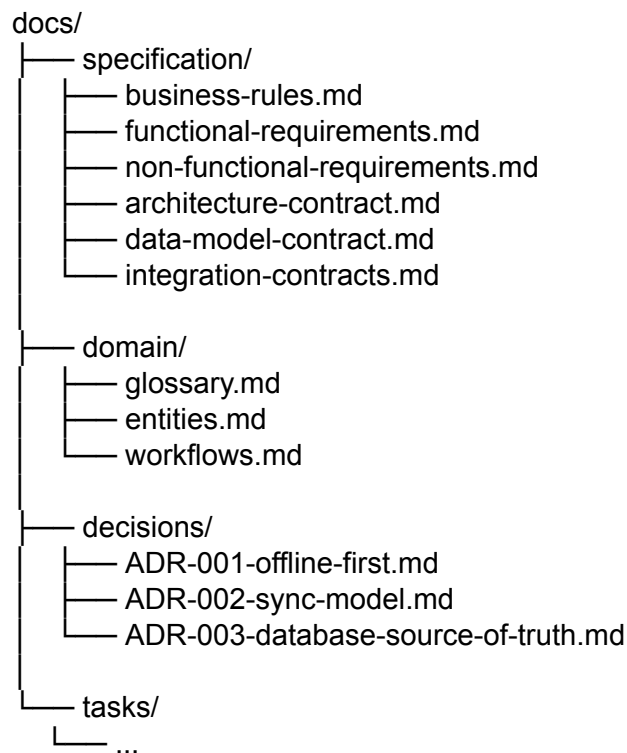


This directly matches the problem you've had.

What I recommend for MBM

Do not make one gigantic MBM_CONTEXT.md.

Instead create a hierarchy:



Then the AI is never allowed to invent business rules when the specification already defines them.

For example:

Sales:

vehicle → party → material → quantity → rate

- gross amount
- cash/bank/GPay
- remaining credit

That is a business contract, not a prompt.

2. One source of truth — but NOT one giant source-of-truth file

This is where your idea about “big companies use one source of truth for variables” needs correction.

The better architecture is:

one authoritative source for each category of truth.

For MBM:

Information	Authoritative source
-------------	----------------------

Database structure	prisma/schema.prisma / migration
Business rules	docs/specification/business-rules.md
Domain terminology	docs/domain/glossary.md
API contract	API schema / validation schema
Type definitions	TypeScript types/generated types
Formatting	Prettier
Naming	ESLint/TypeScript rules
Architecture decisions	ADRs
Agent behavior	AGENTS.md
Tests	automated test suite
Current task	task specification
Deployment	deployment documentation/scripts

This is much better than putting all of it into AGENTS.md.

A June 2026 empirical paper studied 100 repositories with AGENTS.md/CLAUDE.md and found six configuration smells. The most common was Lint Leakage, where developers put rules in the agent context that should already be enforced by linters/formatters. It occurred in 62% of the analyzed files. Context Bloat occurred in 42%. Conflicting Instructions occurred in 28%.

[Configuration Smells in AGENTS.md Files —
arXiv](https://arxiv.org/abs/2606.15828?utm_source=chatgpt.com)

So for your variable naming question:

DON'T

AGENTS.md:

All variables must use camelCase.
Functions must use camelCase.
Classes must use PascalCase.
Constants must use UPPER_CASE.
...

if ESLint/TypeScript can enforce it.

DO

ESLint



enforces naming

AGENTS.md



tells agent to run ESLint and obey existing conventions

Domain glossary



defines what "party", "customer", "vehicle", etc. mean

That is a genuine source-of-truth architecture.

3. AGENTS.md — yes, but keep it small

This is strongly supported by research.

A study of 2,303 agent context files across 1,925 repositories found that developers commonly put:

implementation details — 69.9%

architecture — 67.7%

build/run commands — 62.3%

But security and performance requirements appeared in only 14.5% each.

[Agent READMEs: An Empirical Study of Context Files for Agentic Coding —

arXiv](https://arxiv.org/abs/2511.12884?utm_source=chatgpt.com)

Your root file should therefore be something like:

AGENTS.md

1. Project identity
2. Architecture boundaries
3. Non-negotiable rules
4. Source-of-truth locations
5. Commands
6. Testing requirements
7. Change restrictions
8. Where deeper documentation lives

Not the entire project documentation.

The AGENTS.md ecosystem itself recommends exactly this kind of role: project overview, build/test commands, code style, testing, security, deployment, etc., with nested files for subprojects.

[AGENTS.md official project](https://agents.md/?utm_source=chatgpt.com)

4. Progressive disclosure — this is extremely important for your “RAM.md”

This is one area where your existing idea of having Markdown “RAM” is directionally correct, but it needs to be engineered.

The June 2026 configuration-smells paper found Context Bloat in 42% of analyzed agent configuration files. It specifically recommends moving specialized information out of always-loaded context and loading it when required.

So don't do:

AGENTS.md

↓

500 lines

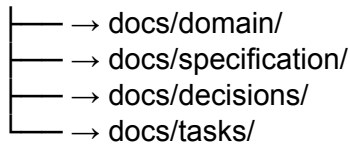
↓

everything about MBM

Instead:

AGENTS.md

|
├── → docs/architecture/



The agent loads the relevant material for the task.

This is called progressive disclosure/context engineering.

The 2025/2026 research on context engineering specifically studied how real projects maintain these persistent agent-context files.

[Context Engineering for AI Agents in Open-Source Software — arXiv](https://arxiv.org/abs/2510.21413?utm_source=chatgpt.com)

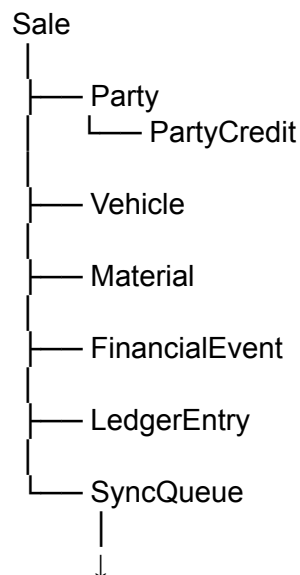
5. Your “mind map” idea → turn it into an architecture graph

This is another thing from your screenshots that is actually research-backed.

A 2025 repository-level code-generation paper represents a codebase as a knowledge graph, capturing inter-file relationships and dependencies so the model retrieves context based on relationships rather than merely searching text. It reported improved repository-level generation compared with its baseline.

[Knowledge Graph Based Repository-Level Code Generation — arXiv](https://arxiv.org/abs/2505.14394?utm_source=chatgpt.com)

For MBM, the graph should conceptually contain:



Supabase

And:

IncomingBoulder



Purchase



Vehicle



Expense / Rent

The purpose isn't to make a pretty diagram.

The purpose is:

> Give the agent structural relationships that are expensive to rediscover by repeatedly searching files.

This is exactly why tools such as the Graphify/codegraph family you showed me are interesting.

6. Task specification before implementation

The March 2026 Context Before Code paper is almost directly about your situation.

The researchers used conversational/vibe coding to build production-oriented systems. They found that AI was effective for scaffolding and routine implementation, but repeatedly failed to preserve architectural constraints when those constraints weren't explicitly defined. They therefore shifted engineering effort toward constraint specification and enforcement auditing.

[Context Before Code — arXiv](https://arxiv.org/abs/2603.11073?utm_source=chatgpt.com)

For MBM, every task should therefore have:

TASK

- Objective
- Scope
- Relevant source-of-truth files
- Existing architecture
- Required behavior
- Forbidden changes
- Acceptance criteria
- Verification commands

└— Stop condition

This matches the workflow you already moved toward:

inspect → plan → edit → test → audit → stop.

That's much more defensible than “here's the feature, go build it.”

7. One task = one bounded change

This is supported indirectly but very strongly by the long-horizon research.

SlopCodeBench, March 2026, is particularly relevant. It tested agents repeatedly extending their own previous implementations under changing requirements.

The results were ugly:

verbosity increased in 89.8% of trajectories

structural erosion increased in 80%

no model solved an entire problem end-to-end across the 11 tested models

agent code became substantially more verbose/eroded than human repositories

prompting improved initial quality but did not stop degradation.

[SlopCodeBench — arXiv](https://arxiv.org/abs/2603.24755?utm_source=chatgpt.com)

That directly validates something you've already experienced:

> 11 individually completed tasks can still produce a worse architecture.

Therefore your rule:

ONE TASK

↓

IMPLEMENT

↓

VERIFY

↓

STOP

is not unnecessarily restrictive.

It is actually a defense against long-horizon architectural erosion.

8. Tests should be written as the agent's objective function

This is probably the strongest practical research result in the entire set.

TDFlow: Agentic Workflows for Test Driven Development, published at EACL 2026, deliberately decomposes repository-scale coding into a test-resolution workflow.

Its architecture separates:

```
graph TD; A[patch proposal] --> B[debugging]; B --> C[patch revision]; C --> D[optional test generation];
```

The system achieved 88.8% on SWE-Bench Lite and 94.3% on SWE-Bench Verified in its reported evaluation. The authors found that the hardest remaining problem was generating good reproduction tests.

[TDFlow — ACL Anthology](https://aclanthology.org/2026.eacl-long.70/?utm_source=chatgpt.com)

For MBM, this means every important requirement should eventually become a test.

Example:

Requirement:

A sale paid partly by cash and partly by GPay must calculate remaining credit correctly.

Becomes:

Test:

sale = ₹10,000

cash = ₹3,000

gpay = ₹2,000

expected:

paid = ₹5,000
credit = ₹5,000

Then the agent doesn't get to argue:

> "The implementation looks correct."

The test answers that.

9. Independent verification should be a separate stage

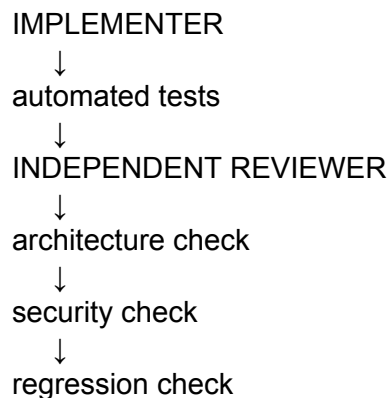
This is one of the biggest changes I would make to your MBM workflow.

Don't let:

Agent A:
"I implemented the feature and all tests pass."

be the final authority.

Use:



Research in July 2026 examining 1.02 million reviewed PRs across 207 GitHub projects found that agent-involved review can accelerate decisions, but the efficiency improvement did not automatically translate into better review quality.

[From Human-Centric to Agentic Code Review —
arXiv](https://arxiv.org/abs/2607.13196?utm_source=chatgpt.com)

And this isn't merely academic. CodeRabbit's August 2026 funding/usage data shows the growing industry around independent AI code review: it reports more than 2 million code

reviews per week across more than 17,000 customers.

So the industry is moving toward:

AI writes → another system checks.

10. Don't trust the agent's own report

This one is surprisingly concrete.

A January 2026 study analyzed 23,247 agent-authored PRs.

They found:

1.7% had high message-code inconsistency

the most common problem was claiming an implementation that wasn't actually implemented

high-inconsistency PRs had 51.7% lower acceptance

they took 3.5× longer to merge.

[Analyzing Message-Code Inconsistency in AI Coding Agent-Authored PRs —
arXiv](https://arxiv.org/abs/2601.04886?utm_source=chatgpt.com)

So your final task report shouldn't be trusted as evidence.

Instead require machine-verifiable evidence:

Files changed:

...

Tests executed:

...

Typecheck:

PASS

Build:

PASS

E2E:

PASS

Relevant regression tests:
PASS

Architecture constraints checked:
PASS

Unrelated files changed:
NO

Known remaining issues:
...

The evidence is the authority.

Not the agent's prose.

11. Refactoring needs its own phase

This is important because AI agents are surprisingly good at local consistency changes but weaker at larger design improvements.

A large empirical study of 15,451 agentic refactoring instances found that agents frequently performed low-level refactorings such as:

Change Variable Type — 11.8%

Rename Parameter — 10.4%

Rename Variable — 8.5%

while higher-level architectural refactoring was less common. Maintainability and readability were the dominant motivations.

[Agentic Refactoring: An Empirical Study of AI Coding Agents —
arXiv](https://arxiv.org/abs/2511.04824?utm_source=chatgpt.com)

For MBM:

Don't ask an implementation agent to simultaneously:

implement feature

+

refactor architecture

+

rename everything
+
clean old code
+
change database structure

That's how you got the sort of problems you previously encountered.

Instead:

PHASE 1
Implement

PHASE 2
Verify

PHASE 3
Architecture audit

PHASE 4
Refactor

PHASE 5
Verify again

12. Static analysis should own style and naming

This is where your “variable naming rules” question gets a very clear answer.

The June 2026 AGENTS.md study specifically calls putting naming/formatting rules into agent context Lint Leakage when those rules can be enforced by automated tools.

Therefore:

Variable naming
↓
ESLint / TypeScript
↓
machine enforced

Formatting
↓
Prettier
↓
machine enforced

Import rules



ESLint



machine enforced

Database naming



Prisma schema + conventions



machine enforced/reviewed

But:

What does "party" mean?

What is "credit"?

What counts as "paid"?

What does "remaining_credit" represent?

What is a "financial event"?

belongs in:

docs/domain/glossary.md

That's the semantic source of truth.

This distinction is extremely important for MBM.

13. Long-term regression testing — SWE-CI

This is the research that most directly explains why your 11-task experience went wrong.

SWE-CI, March 2026, evaluates agents across long-term repository evolution rather than one isolated task.

It contains 100 real repository tasks, each representing roughly 233 days and 71 commits of development history. The benchmark asks whether the agent can maintain a codebase across repeated changes rather than merely fix one issue.

[SWE-CI — arXiv](https://arxiv.org/abs/2603.03823?utm_source=chatgpt.com)

[SWE-CI GitHub
implementation](https://github.com/SKYLENAGE-AI/SWE-CI?utm_source=chatgpt.com)

Your equivalent should be:

MBM regression suite



run before task



agent changes code



run after task



compare



old behavior preserved?

Not just:

new feature works?

but:

old features STILL work?

14. Security needs to be an explicit quality gate

The research on agent-authored refactoring found that 4.7% of modified files introduced new Bandit security findings, even though the PRs were accepted at a high rate.

That means:

tests pass

≠

safe

For MBM, your verification layer should eventually include:

TypeScript

ESLint

Prettier

Prisma validation

Unit tests

Integration tests

E2E

Build

Security scan

Dependency audit

Database migration validation
Sync tests

15. Context files themselves need auditing

This is a new and surprisingly important research direction.

The June 2026 configuration-smell paper identified six problems:

1. Lint Leakage

2. Context Bloat

3. Skill Leakage

4. Conflicting Instructions

5. Init Fossilization

6. Blind References

Across 100 repositories, 91 had at least one of these smells.

That means your future MBM context system itself needs QA.

Every few weeks:

AGENTS.md audit



Are rules still true?



Any duplicated rules?



Any contradictions?



Any rules already enforced by tooling?



Any stale references?



Any task-specific content that should move elsewhere?

This is exactly the opposite of the common mistake:

> “Let's keep adding more instructions because the AI made a mistake.”

That produces an enormous, contradictory AGENTS file.

16. Use a living Architecture Decision Record system

This is the piece I would add specifically because of your previous MBM architectural problems.

Whenever you make an architectural decision:

ADR-001

Offline-first architecture

ADR-002

Supabase synchronization model

ADR-003

Prisma SQLite schema ↔ PostgreSQL schema

ADR-004

AuditLog as sync queue

ADR-005

Financial ledger model

Each:

Decision

Why

Alternatives rejected

Consequences

Status

Then an agent cannot casually decide:

> “I'll replace this architecture because this implementation is easier.”

The agent must detect:

Existing ADR



architectural constraint



needs explicit approval to change

This directly supports your existing rule that architectural decisions require approval.

17. Your “RAM” should be split into three different memories

I would not maintain one massive RAM.md.

Use:

PROJECT TRUTH

docs/specification/

docs/domain/

docs/architecture/

AGENT OPERATING MEMORY

AGENTS.md

TASK MEMORY

docs/tasks/<task-id>.md

And potentially:

CURRENT STATE

docs/status/current-state.md

The distinction:

Permanent truth

What the software is.

business rules

architecture

database model

domain terminology

security rules

Agent instructions

How the agent must behave.

inspect first
one task
no unrelated changes
run verification
architectural changes require approval

Task state

What is happening right now.

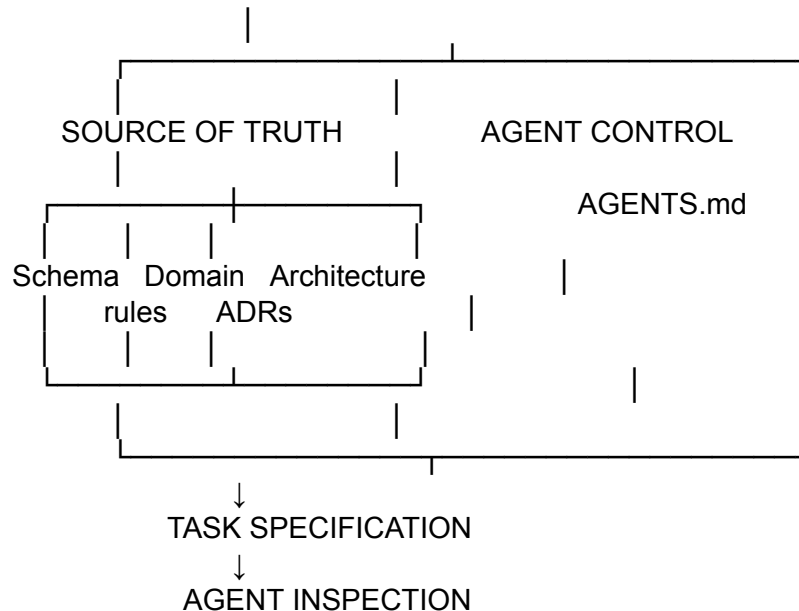
task
attempt
files changed
tests
failures
remaining work

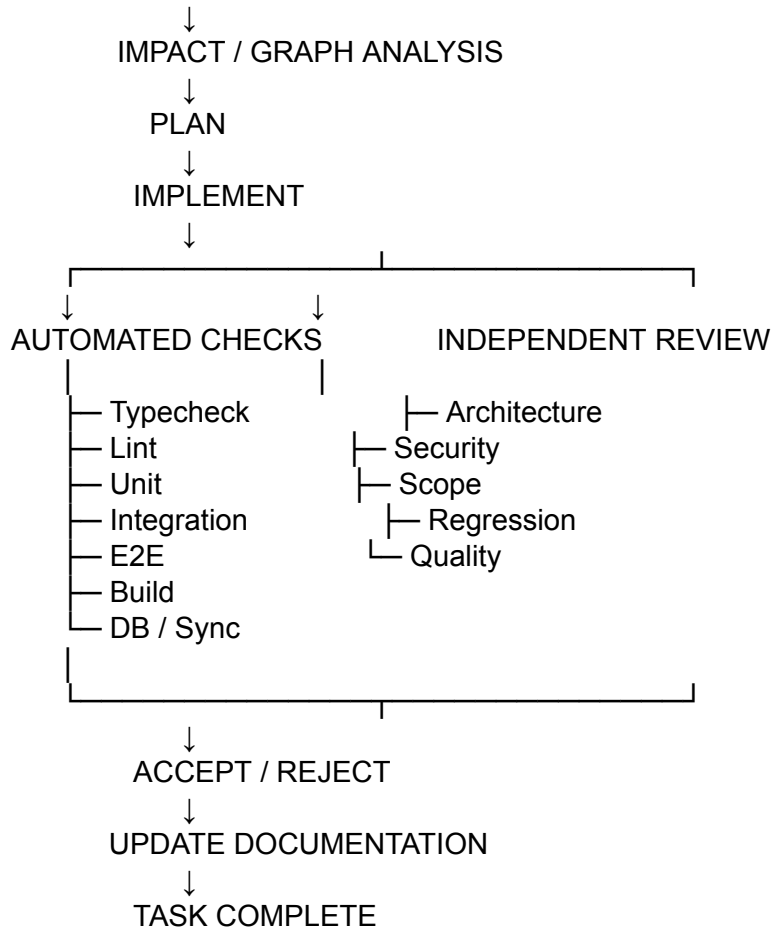
This prevents temporary task information from polluting permanent architecture.

18. Your final MBM architecture should look like this

This is the system I would actually implement.

MBM REPOSITORY





That is the architecture I'd use for a serious pure-vibe MBM workflow.

19. The actual implementation order for MBM

Don't implement all of this simultaneously. Do it in this order.

Set 1 — Source of Truth

Create:

docs/
 └── specification/
 └── domain/
 └── architecture/
 └── decisions/

Then establish:

business rules → specification
database → Prisma/schema
domain terminology → glossary
architecture → ADR

Priority: 10/10

Set 2 — Agent Control

Create a small root:

AGENTS.md

Include only:

- project identity
- architecture boundaries
- source-of-truth map
- commands
- testing requirements
- no-go rules
- task boundaries
- approval requirements

Don't dump your entire MBM documentation into it.

Priority: 10/10

Set 3 — Domain Glossary

Create:

docs/domain/glossary.md

Example:

Party

The customer/entity purchasing material.

Vehicle

A registered transport vehicle used in sales/purchases.

Sale

An outgoing material transaction.

Paid Total

cash + bank + GPay.

Remaining Credit

sale amount - paid total.

This prevents agents from inventing terminology.

Priority: 9/10

Set 4 — Architecture Graph

Use Graphify/codegraph or another repository graph.

Map:

pages

components

services

Prisma models

sync

Supabase

financial system

Priority: 9/10

Research supports the underlying repository-knowledge-graph approach.

Set 5 — Task Contract

Every task gets:

TASK

OBJECTIVE

SCOPE

SOURCE OF TRUTH

IMPACTED AREAS

REQUIREMENTS

NON-REQUIREMENTS

ACCEPTANCE CRITERIA
VERIFICATION
STOP CONDITION

Priority: 10/10

Set 6 — Automated Gate

Before accepting an agent's work:

typecheck
lint
unit tests
integration tests
E2E
build
database validation
sync validation

Priority: 10/10

TDFlow is the strongest research justification here.

Set 7 — Independent Audit

Use a second agent/model after implementation.

Its job is not to rewrite everything.

Its job:

Did the implementation satisfy the specification?

Did it violate architecture?

Did it introduce duplication?

Did it break existing behavior?

Did it create security issues?

Did it modify unrelated files?

Does the implementation match the claimed result?

Priority: 10/10

Set 8 — Regression protection

Create an MBM regression suite and run it after every meaningful change.

The goal changes from:

> “Does this task work?”

to:

> “Does this task work without damaging the existing system?”

SWE-CI and SlopCodeBench are the clearest evidence for why this matters.

Priority: 10/10

Set 9 — Refactoring cycle

After several feature tasks:

feature accumulation



architecture audit



duplication detection



dead-code detection



refactor



full regression

Don't let the feature agent perform unlimited cleanup while implementing features.

Priority: 8/10

Set 10 — Context maintenance

Every significant architecture change should update the relevant source-of-truth document.

Not:

code changed
documentation maybe later

but:

code changed
↓
does architecture/specification change?
↓
YES → update source of truth
NO → continue

The research on AGENTS.md evolution and configuration smells makes this particularly important.

Priority: 8/10

The most important conclusion for your “pure vibe” goal

You can remain a pure vibe coder.

But the architecture should be:

YOU
↓
define intent + constraints
↓
AI
↓
writes implementation
↓
TOOLS
↓

verify objective properties



AI REVIEWER



checks architecture/quality



YOU



approve only important architectural decisions

You do not need to manually understand every line of TypeScript.

But you cannot outsource:

what the system is supposed to mean

architectural boundaries

business rules

source of truth

acceptance criteria

verification policy

That is exactly where the recent research is converging. The March “Context Before Code” study found that production-oriented vibe coding shifted engineering effort from boilerplate toward constraint specification and enforcement auditing. The July SDD paper formalizes the same direction. SlopCodeBench and SWE-CI show why simply letting an agent keep modifying the codebase eventually causes degradation. TDFlow shows how much stronger agents become when constrained by tests.

So for MBM, I would not spend your next effort creating 100 more prompts.

I would build these 10 concrete infrastructure layers:

1. Specification → 2. Domain glossary → 3. Architecture/ADRs → 4. Small AGENTS.md → 5. Task contract → 6. Repository graph → 7. Automated verification → 8. Independent audit → 9. Regression system → 10. Context/source-of-truth maintenance.

That is the evidence-backed version of “pure vibe coding, but with professional software-engineering quality.”

This is gemini research

The Architecture of Autonomous Software Development: Vibe Coding, Compute Arbitrage, and Deterministic Codebase Indexing

The discipline of software engineering is undergoing a foundational paradigm shift driven by the maturity of large language models (LLMs) and the specialized infrastructure surrounding them. This transition is actively moving the industry away from manual syntax generation toward "vibe coding," a methodology where developers leverage natural language to instruct AI agents to generate, test, and deploy entire software architectures. However, the successful implementation of this paradigm is heavily constrained by inherent LLM limitations, primarily token exhaustion, context window degradation, and architectural hallucinations.

To circumvent these bottlenecks, a sophisticated sub-ecosystem of infrastructure has emerged over the past several months. Developers are optimizing the cost of agentic reasoning by exploiting telecommunication promotional loopholes and deploying advanced key rotation proxies. Simultaneously, researchers have achieved unprecedented context compression via algorithms like TurboQuant, radically reducing Key-Value (KV) cache memory constraints. At the repository level, the unreliable nature of semantic vector search (RAG) is being rapidly replaced by deterministic Software Knowledge Graphs (SKGs) that map codebases via Abstract Syntax Tree (AST) parsing.

This comprehensive report details the technical mechanics of these interconnected breakthroughs. It provides an exhaustive analysis of API compute arbitrage, hardware-level vector quantization, codebase indexing engines, strict enterprise variable nomenclature, and visual context management, culminating in a robust architectural implementation plan tailored for large-scale autonomous projects.

The Paradigm of Vibe Coding: A Structural Shift in Developer Disengagement

The term "vibe coding" was formally introduced into the software engineering lexicon by Andrej Karpathy in February 2025, rapidly gaining mainstream recognition, including the Collins English Dictionary Word of the Year. It represents an intent-driven development model where the programmer ceases to manually write boilerplate and instead focuses entirely on testing, architecture refinement, and prompt orchestration.

Empirical research into this phenomenon, such as the 2025 study by Advait Sarkar and Ian

Drosos titled *Vibe coding: programming through conversation with artificial intelligence*, reveals that this methodology fundamentally alters developer cognitive load. Based on rigorous analysis of think-aloud developer sessions, the study determined that vibe coding operates on "iterative goal satisfaction cycles". Developers alternate between high-level conceptual prompting, rapid scanning of generated output, and targeted application testing.

Crucially, the study established that vibe coding does not eliminate the need for engineering expertise; rather, it redistributes it toward "context management" and "material disengagement". The developer's primary role is no longer writing the code, but orchestrating the environment—ensuring the AI has access to a pristine, deterministic context and validating the structural integrity of the output. When the context is flawed, the LLM hallucinates; when API rate limits disrupt the iterative loop, the development cycle collapses. Therefore, establishing a flawless, limitless, and highly structured agentic environment is the prerequisite for effective vibe coding.

LLM Economics: Token Multipliers, API Arbitrage, and Proxy Rotation

The most immediate friction point in autonomous coding is the prohibitive cost and strict rate-limiting of frontier models. Because an autonomous agent (utilizing environments like Cursor, Claude Code, or SillyTavern) must repeatedly ingest thousands of lines of code to maintain state, it rapidly depletes standard API quotas. To sustain the high token throughput required, developers have engineered sophisticated API arbitrage networks that exploit promotional loopholes.

Understanding Nominal Quotas vs. Model Multipliers

A common pitfall in evaluating API aggregators and gateways is the misinterpretation of nominal token quotas. Platforms often advertise massive allocations—such as 7 million or 1 billion free tokens per day—but enforce a "model multiplier" system [cite: User query].

In a multiplier architecture, the consumption rate is weighted against the computational expense of the requested model. If a provider grants a 7 million token nominal quota, a baseline model (e.g., a standard Claude 3.5 Haiku or equivalent) might consume tokens at a 1x rate. However, a frontier reasoning model (e.g., Claude Opus 4.6 or GPT-5.6 Sol) may carry a 4x or 5x multiplier [cite: User query]. Consequently, the effective token allowance is calculated as the nominal quota divided by the multiplier. A 7 million token quota subjected to a 4x multiplier yields only 1.75 million effective tokens [cite: User query]. This hidden constraint explains why "Quality models like opus 4.6 finish fast," as the coding agent's multi-turn reasoning processes burn through the effective limit exponentially faster than anticipated [cite: User query].

Exploiting Promotional Architecture: The Jio AI Pro and Gemini 3 Loophole

Historically, massive compute loopholes existed during beta phases, such as the initial, unmetered rollout of OpenAI's Codex. While those specific vulnerabilities are routinely patched, new promotional architectures continually emerge [cite: User query].

Currently, a significant vector for compute arbitrage revolves around the integration of Google's

Gemini 3 models into regional telecommunication bundles. In India, Reliance Jio launched the "Jio AI Pro" initiative, which bundles 18 months of free access to Google's advanced AI ecosystem for subscribers on unlimited 5G plans (priced at ₹349 and above). This promotion, valued at approximately ₹35,100, unlocks the Gemini 3 Pro model, grants 1,000 advanced AI credits per month, and provides 5 TB of Google One Premium cloud storage.

The structural loophole within this promotion lies in the decoupling of the API access layer from strict single-user identity verification. Because the Jio promotion is activated via the MyJio app and subsequently linked to a Google account, users possessing multiple SIM cards or managing active family plans can generate discrete API access points across multiple Gmail accounts. By manually or programmatically cycling through these provisioned Gmail accounts (e.g., "5 Gmails cycled one after another daily"), a developer can essentially bypass standard rate limits, pooling the cumulative quotas of multiple paid subscriptions to feed a continuous vibe coding agent [cite: 39, User query].

Infrastructure for Automation: Key Rotator Proxies

Manually updating API keys inside an IDE or coding agent is highly inefficient. To automate the extraction of value from these multiple accounts, the open-source community has developed specialized proxy servers designed to sit between the coding agent and the target API provider. These proxies function as local, unified API gateways. They accept standard HTTP requests from the developer's agent, cycle through a database of provisioned API keys, and forward the request to the LLM provider. The leading implementations in this space include:

Proxy Server Repository	Architecture & Language	Core Key Rotation Mechanics	Advanced Features
gemini-proxy (lehuygiang28)	Node.js / Next.js (Fullstack or Serverless)	Intelligent round-robin key rotation with built-in retry mechanisms and exponential backoff upon 429 Too Many Requests errors.	Full UI dashboard, usage tracking per key, OpenAI-compatible endpoint translation, streaming response optimization, Cloudflare Workers support.
geminiproxy (ChakshuGautam)	Go (Golang)	Lightweight, sequential round-robin rotation reading from a simple gemini.keys text file.	Dockerized, highly performant, transparent to clients (compatible with the go-genai client library and standard cURL requests).
gemini-api-key-rotator (jwadow)	Python (FastAPI)	Automatically disables failing keys temporarily with exponentially increasing delays to respect provider-side rate limit cooling periods.	Dual-mode operation (Native Gemini API and OpenAI-compatible endpoints), HTTP/SOCKS5 proxy routing to obfuscate IP origins, SSE streaming support.
gemini-poise (alterem)	Python (FastAPI) & React	Implements a sophisticated Token	Full-stack architecture, SQLAlchemy database

Proxy Server Repository	Architecture & Language	Core Key Rotation Mechanics	Advanced Features
		Bucket algorithm backed by Redis to manage request capacity, current tokens, and refill rates across high/medium/low priority tasks.	management, layered caching (SCAN over KEYS to prevent Redis blocking), user authentication, and pipeline batching.

By deploying one of these proxies (e.g., `gemini-api-key-rotator-proxy-server` via Docker on `localhost:8000`), the developer abstracts the rate limits entirely. The coding agent perceives a single, infinite-compute LLM endpoint, while the proxy silently multiplexes the traffic across the cluster of promotional accounts. Furthermore, because tools like `gemini-proxy` and `ccx` feature OpenAI-compatible endpoint translation, they seamlessly integrate with standard interfaces like SillyTavern, Cursor, or Aider, which typically default to OpenAI formatting.

Solving the Context Bottleneck: The TurboQuant Breakthrough

While API arbitrage solves the financial and rate-limiting constraints of long-running agents, the fundamental hardware bottleneck of inference remains. Passing an entire repository into an LLM prompt forces the model to store massive amounts of key and value vectors in its KV cache to facilitate autoregressive decoding. As context lengths push toward 1 million tokens, the KV cache size grows linearly, eventually exhausting GPU VRAM and crippling inference speed. In 2025, researchers at Google Research and DeepMind (Zandieh, Daliri, Hadian, and Mirrokni) published *TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate*, detailing a revolutionary algorithm that compresses KV cache memory by up to six times (reaching 3 bits per value) with zero accuracy loss.

The Mathematical Mechanics of TurboQuant

Prior vector quantization methods, such as asymmetric 2-bit quantization used in the KIVI baseline, achieved roughly 2.6x compression but suffered from visible quality degradation and lacked mathematical headroom. Other approaches, like Nvidia's KVTC, achieved high compression (20x) but required highly specific, offline model calibration steps. TurboQuant bypasses these limitations by acting as a "data-oblivious" algorithm that requires no per-model calibration. It achieves this through a two-stage companion pipeline:

- PolarQuant (High-Quality Compression):** The algorithm begins by randomly rotating the input data vectors in high-dimensional Euclidean space. This geometric manipulation induces a highly concentrated Beta distribution on the coordinates. The algorithm pairs coordinates, maps them onto a polar coordinate system, applies recursive polar transforms on the radii, and optimally quantizes only the angles using Lloyd-Max scalar quantizers. This stage eliminates the need to store per-block, full-precision normalization constants, drastically dropping overhead to zero while using the majority of the bit budget to capture the core vector concept.

2. **QJL Transform (Residual Error Correction):** Following the MSE quantization, a small residual error remains. TurboQuant utilizes a 1-bit Quantized Johnson-Lindenstrauss (QJL) transform to act as a mathematical error-checker. The QJL step stores a sign vector for the residual. This effectively removes any mathematical bias, guaranteeing optimal, unbiased inner-product estimation for attention-logit calculation.

The resulting performance metrics are paradigm-shifting for coding agents. Benchmarks on Nvidia H100 GPUs indicate up to an eightfold improvement in attention-logit computation speed. Across rigorous long-context evaluations (LongBench, Needle In A Haystack, ZeroSCROLLS), TurboQuant maintained perfect retrieval scores from 4,000 up to 104,000 tokens at 3.5-bit compression, mirroring full 32-bit baseline performance.

For the autonomous software developer, this implies that models can natively ingest massive repositories, including exhaustive business rules, PR histories, and SQL schemas, continuously without latency spikes or memory failures, thereby enabling true repository-scale reasoning.

Deterministic Codebase Indexing: The Software Knowledge Graph (SKG)

Despite the advent of massive context windows facilitated by algorithms like TurboQuant, relying purely on raw file ingestion via "grep" or vector semantic search (RAG) is inherently inefficient for codebases. Code is highly structured; it operates on rigid architectural relationships, cross-file imports, class inheritance, and strict variable scoping. Probabilistic vector databases frequently fail to distinguish between two identically named functions residing in different modules, leading to agent hallucinations.

The industry is currently transitioning away from RAG toward deterministic Software Knowledge Graphs (SKGs). Two prominent systems leading this shift are Graphify and CODENS.

Graphify: AST-Driven Local Graph Extraction

Graphify is a command-line utility and IDE skill that parses an entire repository—including source code, Markdown documentation, SQL schemas, and multimodal assets (PDFs, images)—into a queryable graph database (graph.json).

Instead of relying on LLM embeddings, Graphify employs tree-sitter, a deterministic Abstract Syntax Tree parser capable of analyzing over 40 programming languages locally on the developer's machine. The system maps structural relationships (e.g., calls, imports, inherits, mixes_in) and tags them in the graph with an EXTRACTED confidence label, explicitly denoting that the relationship is a hard coded fact. Simultaneously, the tool executes a semantic pass over inline comments (such as # NOTE: or # HACK:) and architecture documentation, linking these contextual rationales to their respective code nodes with an INFERRED tag.

When an AI coding agent (e.g., Claude Code, Cursor, or Antigravity) requests context, a PreToolUse hook intercepts the agent's standard file-search command and forces it to query the Graphify engine first. The agent can execute commands such as graphify path A B to instantly trace the shortest path of dependency between two isolated components, or view "God nodes" (centralized modules through which the entire architecture flows) determined by Leiden community detection algorithms.

By providing a pre-indexed, structural map of the repository, Graphify drastically reduces the tokens burned during exploratory file reading. Some benchmarks suggest an up to 71.5x

improvement in token-efficiency for targeted architectural queries [cite: User query, 26, 31].

CODENS: Graphing Temporal Evolution

While Graphify excels at static snapshot analysis, large projects often suffer from undocumented historical context. CODENS addresses this by building a typed software knowledge graph derived sequentially from pull requests (PRs).

Evaluated on complex Ruby on Rails production systems, CODENS first maps skeleton nodes based on architectural conventions (Controllers, Models, Services). It then chronologically replays the repository's Git history. For every pull request, CODENS injects the PR diff and the prior state of the node into an LLM, prompting it to extract schema-driven semantic updates (e.g., business logic changes, behavioral flows) and merge them into a Neo4j database.

This transforms code changes into "living, accessible, and queryable documentation". Coding agents are provided with explicit tool calls (GET_NODE, GET_NEIGHBORS, CYPHER) to traverse this historical graph, allowing them to answer deeply nuanced repository-level questions, such as why a specific authentication middleware was altered six months prior.

Vibe Coding Specifications: Standards and Visual Working Memory

Providing an AI with a massive, compressed context window and a deterministic knowledge graph is insufficient if the underlying codebase logic is obfuscated by poor human architecture. Vibe coders must architect their projects specifically for machine comprehension. This requires rigid adherence to variable nomenclature, strict state management, and the provision of persistent visual memory.

Variable Naming: The Enterprise "Rule of Two"

AI coding agents are highly sophisticated pattern-matching engines. When variable naming conventions are inconsistent, the agent misinterprets state, leading to logic drift during autonomous generation. Establishing strict enterprise coding standards—such as the "Rule of Two"—is imperative.

The general principles of variable nomenclature for AI ingestion dictate that names must be unequivocally descriptive, avoiding shorthand or single-character identifiers outside of localized loop iterations (i, j).

- **Case Consistency:** Python environments must strictly utilize snake_case (e.g., user_age), while C-based or JavaScript frameworks typically mandate camelCase (e.g., myVariableName).
- **Constant Immutability:** Variables acting as constants must be defined using "screaming snake case" (MAX_VALUE) to visually and programmatically signal to the LLM that the value should not be targeted for reassignment during refactoring tasks.
- **Scope Isolation:** The LEGB (Local, Enclosing, Global, Built-in) hierarchy dictates variable accessibility. To prevent an agent from inappropriately exposing internal variables during public API generation, developers must enforce the Python convention of prefixing non-public variables with a single underscore (e.g., _timeout).

Monorepos and The Single Source of Truth

In distributed architectures, vibe coders must eliminate configuration fragmentation. If environment variables or core configuration logic are duplicated across multiple files, the coding agent will inevitably update one reference while ignoring the others, breaking the system state. The resolution is the enforcement of a "Single Source of Truth" (SSOT) architecture, commonly achieved by migrating discrete services into a Monorepo orchestrated by tools like Turborepo. Turborepo acts as the execution brain, providing the agent with standardized task definitions (e.g., executing `bun dev` to simultaneously launch frontend, backend, and agentic services). Furthermore, all environment variables must be decoupled from the code and centralized within a secrets management tool like Doppler or Infisical. By executing commands via dynamic injection (e.g., `infisical run --env=dev -- npm run dev`), the codebase remains entirely devoid of hardcoded configuration drift, ensuring the AI agent interacts with a pristine, immutable set of operational parameters validated by strict schema libraries like zod-env.

Mermaid Mindmaps as "Markdown RAM"

A critical challenge in agentic coding is maintaining the AI's spatial and architectural understanding across long, multi-turn sessions. Expert vibe coders resolve this by establishing "Markdown RAM"—a persistent `.md` file containing detailed Mermaid diagrams that the agent uses as a visual working memory.

By continuously updating a flowchart or mindmap after every structural change, the agent maintains a grounded schematic of the system state. However, to ensure these diagrams render correctly within GitHub or IDE preview panes, explicit `.cursorrules` (custom instructions for the agent) must be enforced:

Rule	Specification	Rationale for AI Constraints
Orientation Preference	Default to graph TD (Top-Down) layouts.	Vertical layouts render with substantially greater readability within narrow IDE markdown panes compared to LR (Left-Right) orientations.
Node ID Sanitization	Mandate strict alphanumeric IDs without spaces (e.g., <code>userAuth</code>).	Spaces in Node IDs trigger frequent parsing failures. Any textual labels containing spaces must be encapsulated in quotation marks (e.g., <code>A["Validate Data"]</code>).
Mindmap Hierarchy	Enforce spatial indentation exclusively for mindmaps.	GitHub's native Mermaid integration relies on basic indentation. Injecting custom icons (e.g., <code>::icon(fa fa-one)</code>) will instantly break the rendering pipeline.
Styling Restrictions	Prohibit the generation of custom themes or <code>classDef</code> CSS directives.	GitHub and Obsidian completely ignore inline Mermaid theme directives,

Rule	Specification	Rationale for AI Constraints
		overriding them with the native application's light/dark mode settings. AI attempts to colorize nodes waste tokens and generate bloated code.
Service Grouping	Utilize the subgraph syntax to delineate architectural boundaries.	Platforms like Whimsical and tldraw natively translate Mermaid subgraph syntax into interactive, grouped geometric shapes, allowing for seamless transition from code to interactive whiteboard.

Implementation Blueprint: The "Pure Vibe" MBM Architecture

For developers aiming to leverage this infrastructure—such as orchestrating the MBM project described in the original query—a strict, sequential integration plan is required. This blueprint synthesizes the scattered research into a highly cohesive operational environment.

Phase 1: Establish the Compute Arbitrage Network

The foundation requires securing high-volume, resilient API access without incurring prohibitive costs.

- Activate Promotional Tiers:** Provision multiple accounts taking advantage of the Jio AI Pro / Gemini 3 18-month free tier and Alibaba's 70M+ token (Qwen3.8-Max) allocations [cite: 74, User query].
- Deploy the API Gateway:** Initialize a Dockerized instance of gemini-poise or gemini-api-key-rotator-proxy-server locally. Populate the proxy's credential store (gemini.keys) with the activated promotional API keys.
- Agent Configuration:** Configure the primary coding agent (e.g., Cursor, SillyTavern, or Antigravity) to route all outbound LLM traffic to the local proxy endpoint (e.g., `http://127.0.0.1:8000/v1`) using the OpenAI-compatible formatting protocol. Ensure the agent's escalation model is set to a highly capable frontier reasoning engine (e.g., Gemini 3 Pro or GLM-5.3) for complex tasks [cite: User query].

Phase 2: Index the Knowledge Graph

Once infinite compute is secured, the repository must be parsed deterministically to prevent RAG-induced hallucinations.

- Initialize Graphify:** Install the engine via `uv tool install graphify` and execute `graphify install` to register the PreToolUse intercept hooks within the local IDE.
- Generate the Graph:** Execute `/graphify .` at the root of the MBM repository to generate the baseline `graph.json`. The AST parser will recursively map all files, function dependencies, and imported schemas.
- Implement CODENS Temporal Tracking:** For deeper historical context, initialize a Neo4j

instance and configure CODENS to parse the PR history of the repository, extracting schema-driven logic updates to build a living documentation layer.

Phase 3: Enforce Structural Standards and Markdown RAM

The final phase requires conditioning the AI to strictly adhere to enterprise architecture and visual memory protocols.

1. **Centralize Variables:** Migrate the MBM project into a Turborepo monorepo architecture. Abstract all environmental parameters into Infisical, eliminating hardcoded variables and establishing a strict single source of truth.
2. **Establish .cursorrules:** Implement a rigid rule set mandating the "Rule of Two" variable nomenclature (strict casing, descriptive phrasing, underscore scoping) to ensure the AI generates predictable syntax.
3. **Activate Visual Memory:** Create an architecture.md file within the repository root. Instruct the agent via system prompts to continuously update the Mermaid diagrams within this file (graph TD layout, strict node sanitization, no custom CSS) after completing any structural engineering task. This guarantees the agent maintains a flawless, persistent understanding of the evolving MBM architecture.

Conclusion

The architecture of modern autonomous software development requires moving far beyond simplistic prompt engineering. The integration of high-availability proxy networks neutralizes the economic and operational constraints of API rate limits, while compression algorithms like TurboQuant mathematically guarantee the preservation of massive context windows. By transitioning away from probabilistic vector search toward the deterministic, AST-driven reality of Graphify and CODENS, developers eliminate the primary vectors for AI hallucination. When deployed within a rigidly defined structural environment—characterized by single-source-of-truth configurations and persistent Mermaid-based visual memory—the vibe coder is empowered to oversee complex, repository-scale evolution with total deterministic control.

This is for informational purposes only. For medical advice or diagnosis, consult a professional.

Works cited

1. Vibe coding - Wikipedia, https://en.wikipedia.org/wiki/Vibe_coding 2. What is Vibe Coding? - IBM, <https://www.ibm.com/think/topics/vibe-coding> 3. What is vibe coding? - Cloudflare, <https://www.cloudflare.com/learning/ai/ai-vibe-coding/> 4. New Gemini banwave ? : r/SillyTavernAI - Reddit, https://www.reddit.com/r/SillyTavernAI/comments/1n014d0/new_gemini_banwave/ 5. Jio Brings Google's Gemini 3 AI Model to All 5G Unlimited Subscribers: Check Eligibility, Benefits | Technology News - Gadgets 360, <https://www.gadgets360.com/ai/news/jio-google-gemini-3-rollout-free-offer-eligibility-availability-9661131> 6. ChakshuGautam/geminiproxy: A gemini proxy rotating keys for people with multiple accounts. - GitHub, <https://github.com/ChakshuGautam/geminiproxy> 7. TurboQuant - Wikipedia, <https://en.wikipedia.org/wiki/TurboQuant> 8. How Google's TurboQuant Compresses LLM Memory by 6x (With Zero Accuracy Loss), <https://www.buildfastwithai.com/blogs/google-turboquant-kv-cache-6x-compression> 9. GitHub -

Graphify-Labs/graphify: Turn any codebase, with its docs, SQL schemas, configs, and PDFs, into a queryable knowledge graph. A /graphify skill for Claude Code, Cursor, Codex, and Gemini CLI: local deterministic AST parsing, every edge explained, no vector store., <https://github.com/Graphify-Labs/graphify> 10. CODENS: Transforming Code Changes into Living, Accessible, and Queryable Documentation - arXiv, <https://arxiv.org/html/2607.18356v1> 11. [2506.23253] Vibe coding: programming through conversation with artificial intelligence, <https://arxiv.org/abs/2506.23253> 12. Brioch/gemini-openai-proxy - GitHub, <https://github.com/Brioch/gemini-openai-proxy> 13. How to Use SillyTavern: A Practical Guide to Setup, Models, and Advanced RP Features, <https://sider.ai/blog/ai-tools/how-to-use-sillytavern-a-practical-guide-to-setup-models-and-advanced-rp-features> 14. lehuygiang28/gemini-proxy: A scalable proxy for the Gemini API that handles API key rotation, polling, and load balancing. Perfect for Next.js, Cloudflare Workers, and serverless environments - GitHub, <https://github.com/lehuygiang28/gemini-proxy> 15. Gemini Poise: Gemini API Proxy and Management Tool - GitHub, <https://github.com/alterem/gemini-poise> 16. Google Gemini Offer from Jio: a partnership transforming student life in India, <https://www.jio.com/jiostories/jio-google-gemini/> 17. Google Gemini Offer: Free 18-Month Pro Plan for Jio Users, <https://www.jio.com/google-gemini-offer/> 18. Gemini API Key Rotator Proxy Server - GitHub, <https://github.com/jwadow/gemini-api-key-rotator-proxy-server> 19. GitHub - BenedictKing/ccx: Claude / Codex / Gemini API Proxy, <https://github.com/BenedictKing/ccx> 20. TurboQuant.net - Independent TurboQuant Analysis, <https://turboquant.net/> 21. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate - arXiv, <https://arxiv.org/pdf/2504.19874> 22. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate - OpenReview, <https://openreview.net/forum?id=tO3ASKZlok> 23. TurboQuant: Redefining AI efficiency with extreme compression - Google Research, <https://research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/> 24. Graphify: Building a Knowledge Graph of Your Codebase for AI Assistants - Better Stack, <https://betterstack.com/community/guides/ai/ai-development/graphify-codebase/> 25. CodexGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases | Request PDF - ResearchGate, https://www.researchgate.net/publication/392505894_CodexGraph_Bridging_Large_Language_Models_and_Code_Repositories_via_Code_Graph_Databases 26. Graphify Labs - GitHub, <https://github.com/Graphify-Labs> 27. graphify - AI Agents on GitHub (106.8k) | SkillsLLM, <https://skillsllm.com/skill/graphify-labs-graphify> 28. [2607.18356] CODENS: Transforming Code Changes into Living, Accessible, and Queryable Documentation - arXiv, <https://arxiv.org/abs/2607.18356> 29. Python Variables: Concepts, Best Practices, and Mechanics | by Learn to code with tara, <https://medium.com/@tharannumcode/python-variables-concepts-best-practices-and-mechanics-c4fdf76834c0> 30. GitHub - learnwithfair/c-programming-documentation, <https://github.com/learnwithfair/c-programming-documentation> 31. Repository Structure | JSON Resume Docs, <https://jsonresume.org/docs/002-repository-structure> 32. Scaling My Development with Git Worktrees and Agentic Workflows - Medium, <https://maxlang-71139.medium.com/scaling-my-development-with-git-worktrees-and-agentic-workflows-f30e688cbd72> 33. Terraform monorepo vs. multi-repo: The great debate - HashiCorp, <https://www.hashicorp.com/en/blog/terraform-mono-repo-vs-multi-repo-the-great-debate> 34. podcast-guest-crm by RudrenduPaul | Glama, <https://glama.ai/mcp/servers/RudrenduPaul/podcast-guest-crm> 35. kasperjunge/vibe-coding.dk | DeepWiki, <https://deepwiki.com/kasperjunge/vibe-coding.dk> 36. Brainstorm and Planning with

Mermaid Diagrams in Cursor - Egghead.io,
<https://egghead.io/brainstorm-and-planning-with-mermaid-diagrams-in-cursor~9iu69> 37.
Working with Mermaid code in Whimsical, <https://whimsical.com/learn/boards/mermaid>